

Pipeline Microsoft Fabric : lister, filtrer, charger, mémoriser

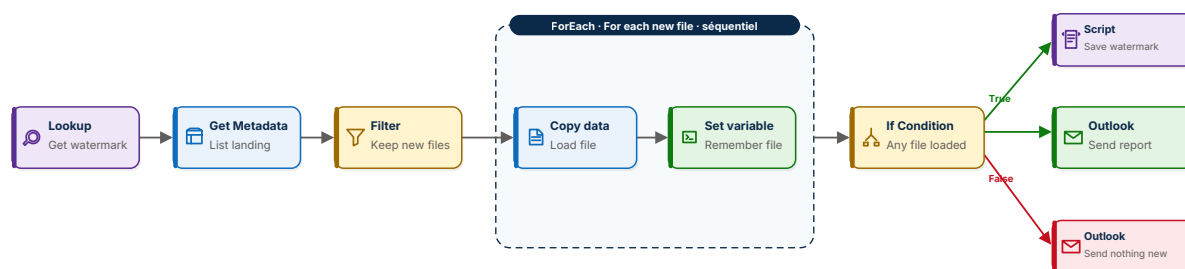
LISTER

FILTRE

CHARGER

MÉMORISER

Des capteurs déposent chaque jour un fichier de relevés dans le lakehouse. Le pipeline lit où il s'était arrêté, liste le dossier, ne garde que les fichiers nouveaux, les charge, puis note le dernier fichier traité dans un entrepôt. Relancé dix fois, il ne recharge jamais rien. Dix activités, aucune ligne de code, et un pipeline qui se souvient.



PL_ColdChain_Express. Violet : la mémoire du pipeline, lue au départ et écrite à l'arrivée. Fil vert : des fichiers ont été chargés. Fil rouge : rien de nouveau, on le dit.

10
activités

9
types d'activité

0
ligne de code

0
fichier relu deux fois

7
pages

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

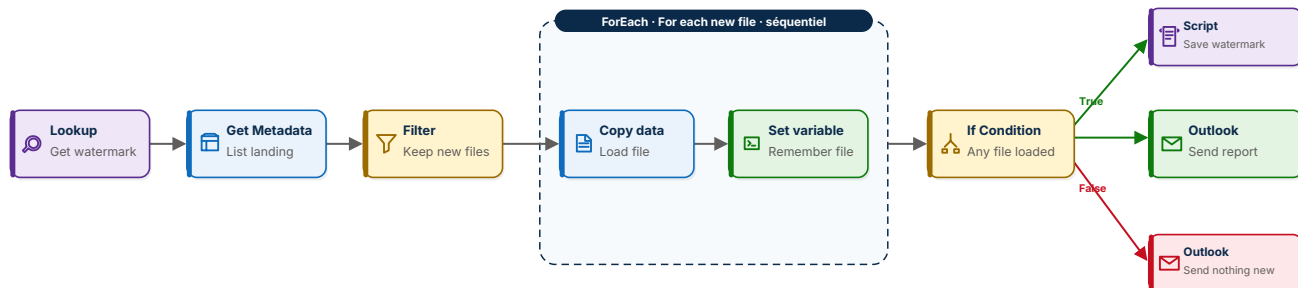
PRÉREQUIS
Un espace de travail Fabric sur une capacité, une boîte Outlook Microsoft 365

FICHIERS DU PROJET
github.com/abdoulhamiddiallo/fabric-pipeline-express-3

ÉDITION
Octobre 2026

Ce que vous construisez

Trois entrepôts frigorifiques, Woippy, Ennery et Augny, relèvent la température de leurs zones frais et surgelés toutes les trente minutes. Chaque jour, un fichier `readings_<date>.csv` arrive dans `Files/landing`. Le pipeline ne reçoit aucune date en paramètre : il sait seul ce qu'il a déjà chargé. C'est le chargement incrémental par filigrane, la technique que tout pipeline de production finit par utiliser.



Dix activités, neuf types. Violet : lecture et écriture de la mémoire. Bleu : fichiers et métadonnées. Jaune : décision. Vert : réussite. Rouge : rien à faire. Dans ce guide, les libellés **en bleu** sont les mots exacts de l'interface Fabric, et les blocs bleu clair se copient tels quels.

● Un filigrane, pas un paramètre

Le nom du dernier fichier chargé vit dans une table d'un entrepôt. Le pipeline le lit au départ, l'écrit à l'arrivée. Personne ne lui dit quoi charger.

● Filtrer avant de boucler

Le dossier est listé en entier, puis une activité Filter ne garde que les CSV dont le nom dépasse le filigrane. La boucle ne voit que du nouveau.

● Relancer ne coûte rien

Pas de fichier nouveau : la boucle tourne zéro fois, la mémoire ne bouge pas, un e-mail le dit. La table `readings` ne reçoit jamais deux fois la même ligne.

CE QUE VOUS SAUREZ FAIRE

Un chargement incrémental qui ne dépend d'aucun paramètre, reprend seul après un échec et ne relit jamais un fichier.

CE QUE VOUS AUREZ PRATIQUÉ

Lookup sur entrepôt, Get Metadata, Filter, ForEach, Copy en Append, Set variable, If Condition, Script, Outlook.

Les dix activités

ACTIVITÉ	TYPE	RÔLE
Get watermark	Lookup	Lit dans l'entrepôt le nom du dernier fichier chargé
List landing	Get Metadata	Liste les fichiers présents dans <code>Files/landing</code>
Keep new files	Filter	Ne garde que les <code>.csv</code> dont le nom est postérieur au filigrane
For each new file	ForEach	Parcourt les fichiers retenus, en séquentiel
Load file	Copy data	Ajoute le fichier à la table <code>readings</code> , avec la colonne <code>source_file</code>
Remember file	Set variable	Mémore le nom du fichier qui vient d'être chargé
Any file loaded	If Condition	Au moins un fichier retenu ?
Save watermark	Script	Branche True : écrit le nouveau filigrane dans l'entrepôt
Send report, Send nothing new	Office 365 Outlook	Un e-mail avec le bilan, ou un e-mail « rien de nouveau »

Le jeu de données

FICHIER	LIGNES	ALERTES
<code>readings_2026-10-04.csv</code>	1 152	2
<code>readings_2026-10-05.csv</code>	1 152	11
<code>readings_2026-10-06.csv</code>	1 140	0
<code>readings_2026-10-07.csv</code>	1 152	6
<code>readings_2026-10-08.csv</code>	1 152	7

Cinq journées dans `data/`, un fichier par jour. Vingt-quatre capteurs (3 sites × 2 zones × 4 capteurs), un relevé toutes les trente minutes : 1 152 lignes par jour. Le 6 octobre, un capteur d'Ennery est resté muet six heures : 12 lignes de moins. Neuf colonnes : `reading_id`, `reading_time`, `site`, `zone`, `sensor_id`, `temperature_c`, `humidity_pct`, `door_open`, `status`. Une alerte est un relevé hors plage : au-dessus de 4 °C en frais, au-dessus de -18 °C en surgelés. Vous déposerez trois fichiers avant la première exécution, puis les deux autres plus tard : c'est ainsi que vous verrez le filigrane travailler.

Préparer, lire la mémoire, lister, filtrer

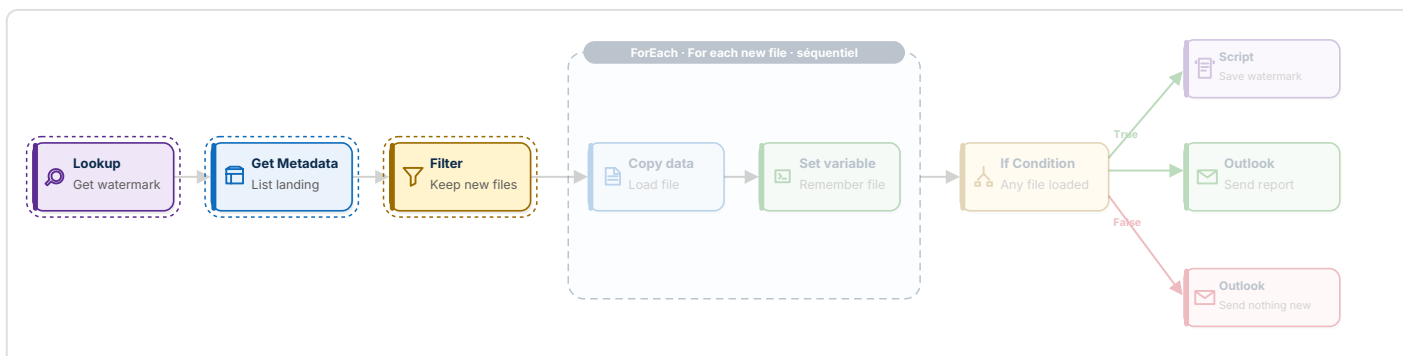
1 · LIRE ET LISTER

2 · FILTRER

3 · CHARGER

4 · MÉMORISER

5 · EXÉCUTER



1 Les objets Fabric

- 1 Créez un lakehouse **LH_ColdChain**. Dans **Fichiers**, **Nouveau sous-dossier** landing, puis **Charger**, **Charger des fichiers** : déposez-y les trois premiers fichiers seulement, du 4, 5 et 6 octobre.

- 2 Créez un entrepôt **WH_Control** (Warehouse). **Nouvelle requête SQL**, exécutez :

```
CREATE TABLE dbo.watermark (
  pipeline VARCHAR(100) NOT NULL,
  last_file VARCHAR(200) NOT NULL,
  updated_at DATETIME2(6) NOT NULL;
INSERT INTO dbo.watermark
VALUES ('PL_ColdChain_Express', '', GETDATE());
```

- 3 Créez le pipeline **PL_ColdChain_Express**. Fond du canevas, onglet **Variables**, **Nouveau** :

Nom	last_file
Type	String
Valeur par défaut	vide

POURQUOI UN ENTREPÔT Un lakehouse se lit très bien mais ne s'écrit pas en SQL depuis un pipeline. L'entrepôt, lui, accepte un UPDATE : c'est la seule raison de sa présence, et c'est exactement ce qu'il faut pour une mémoire d'une ligne. DATETIME2(6) et non DATETIME2 : l'entrepôt refuse la précision 7.

UNE HABITUDE QUI SAUVE UNE SOIRÉE Enregistrer après chaque activité terminée, et vérifier que le point à côté du nom du pipeline a disparu.

2 Get watermark

LOOKUP

- 1 Ajoutez **Rechercher** (Lookup), nom Get watermark.
- 2 **Paramètres** : connexion **Entrepôt**, espace de travail courant, WH_Control. **Utiliser la requête** : **Requête** :

```
SELECT last_file FROM dbo.watermark
WHERE pipeline = 'PL_ColdChain_Express'
```

- 3 Laissez **Première ligne uniquement** coché. **Aperçu des données** : une ligne, last_file vide.

CE QUE RENVOIE L'ACTIVITÉ output.firstRow.last_file. Vide aujourd'hui ; demain, le nom du dernier fichier chargé.

3 List landing

GET METADATA

- 1 Ajoutez **Obtenir les métadonnées**, nom List landing, relié à la sortie verte du Lookup.
- 2 **Paramètres** : **Lakehouse**, LH_ColdChain, **Fichiers**, chemin du dossier landing, nom de fichier vide. Format **Binaire**.

- 3 **Liste des champs**, **Nouveau** : **Éléments enfants**.

CE QUE RENVOIE L'ACTIVITÉ output.childItems, un tableau d'objets { name, type }, un par fichier du dossier, dans l'ordre des noms.

4 Keep new files

FILTER

- 1 Ajoutez **Filtrer**, nom Keep new files, relié à la sortie verte de List landing.

- 2 **Éléments** :

```
@activity('List landing').output.childItems
```

- 3 **Condition** :

```
@and(endswith(item().name, '.csv'),
greater(item().name, activity('Get
watermark').output.firstRow.last_file))
```

LE CŒUR DU PROJET Comparer des noms suffit, parce que les noms portent la date : readings_2026-10-07.csv est plus grand que readings_2026-10-06.csv. Sortie : output.Value (les fichiers retenus) et output.FilteredItemsCount.

À l'intérieur de la boucle : charger, mémoriser

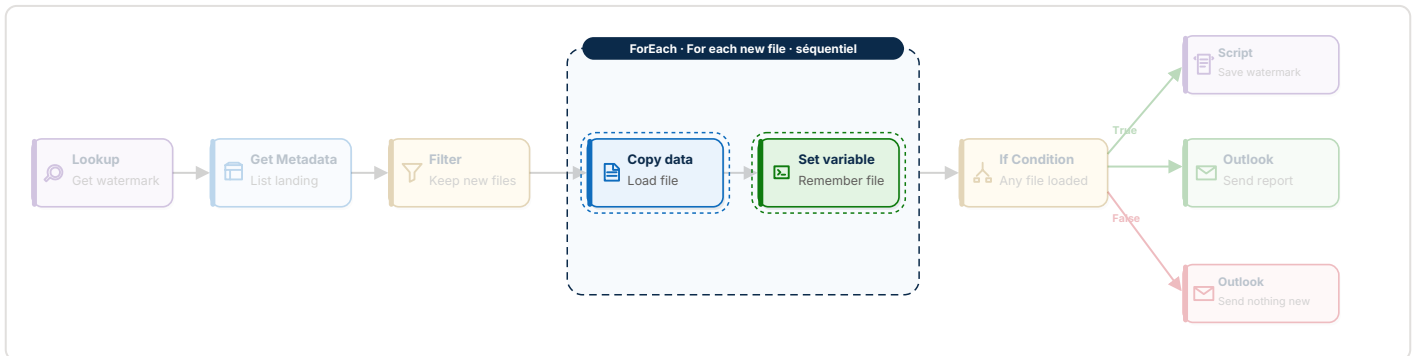
1 · LIRE ET LISTER

2 · FILTRER

3 · CHARGER

4 · MÉMORISER

5 · EXÉCUTER



5 For each new file

FOREACH

1 Ajoutez **ForEach**, nom For each new file, relié à la sortie verte de Keep new files.

2 **Paramètres** : cochez **Séquentiel**. **Éléments** :

```
@activity('Keep new files').output.Value
```

3 Cliquez sur le crayon : vous entrez dans la boucle. Le fil d'Ariane vous ramène au pipeline.

SÉQUENTIEL, ET NON PARALLÈLE La boucle écrit dans la variable `last_file` à chaque tour. En parallèle, le dernier à écrire ne serait pas le dernier fichier. En séquentiel et dans l'ordre des noms, la variable finit avec le fichier le plus récent. Dans la boucle, `item().name` est le nom du fichier courant.

POURQUOI APPEND, ET POURQUOI C'EST SANS DANGER

Beaucoup de pipelines reconstruisent leur table à chaque exécution, ce qui est sûr mais relit tout. Ici la table grandit, et c'est le filigrane qui garantit qu'un fichier n'y entre qu'une fois. La colonne `source_file` en est la preuve : `COUNT(DISTINCT source_file)` dira toujours combien de fichiers sont passés, jamais plus.

7 Remember file

SET VARIABLE

1 Ajoutez **Définir la variable**, nom Remember file, reliée à la sortie verte de Load file.

2 **Paramètres** : variable `last_file`, valeur :

```
@item().name
```

POURQUOI APRÈS LE COPY, ET NON AVANT Si le Copy échoue, la variable ne prend pas ce nom, le fil vert ne va pas plus loin, et le filigrane ne sera pas avancé. Le fichier fautif sera repris à la prochaine exécution. L'ordre des deux activités est le mécanisme de reprise du pipeline : il ne coûte rien et il est complet.

CE QUE FAIT LA BOUCLE QUAND IL N'Y A RIEN Zéro élément dans `Value` : zéro itération, pas d'erreur, la variable reste vide. C'est ce vide que la condition de l'étape 4 va tester.

6 Load file

COPY DATA

1 Dans la boucle, ajoutez **Copier les données**, nom Load file.

2 **Source** : **Lakehouse**, `LH_ColdChain`, **Fichiers**, format **DelimitedText**, en-tête coché. Les deux cases du chemin :

```
landing
```

```
@{item().name}
```

3 Dépliez **Avancé**, **Colonnes supplémentaires**, **Nouveau** : nom `source_file`, valeur `@item().name`.

4 **Destination** : **Lakehouse**, `LH_ColdChain`, **Tables**, table `readings`, action **Ajouter** (Append).

AVANT DE QUITTER LA BOUCLE

☐ le ForEach est **SÉQUENTIEL**

☐ Load file a sa colonne `source_file` et l'action **AJOUTER**

☐ Remember file est relié derrière Load file, pas devant

☐ Enregistrer, le point a disparu

Décider, écrire la mémoire, prévenir

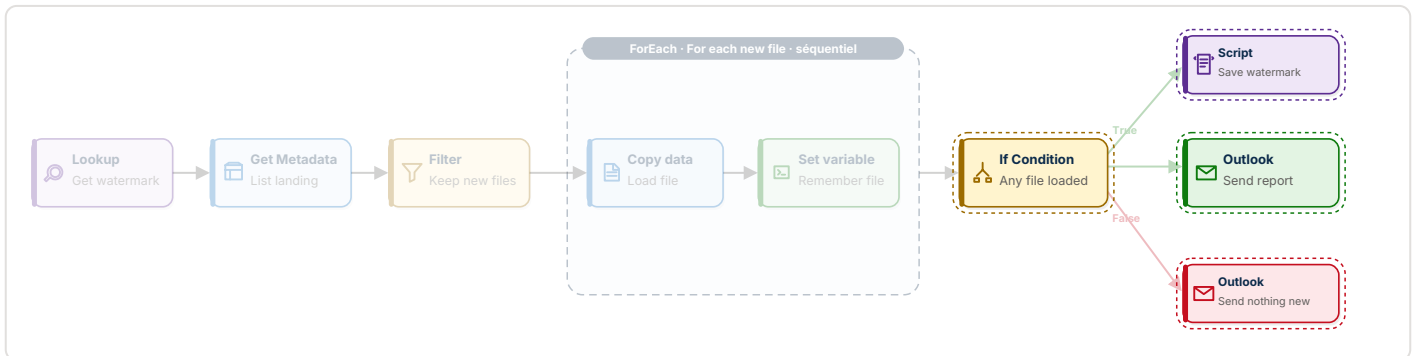
1 · LIRE ET LISTER

2 · FILTRER

3 · CHARGER

4 · MÉMORISER

5 · EXÉCUTER



Revenez au canevas principal par le fil d'Ariane ; les quatre activités suivantes se posent après la boucle.

8 Any file loaded

IF CONDITION

- 1 Ajoutez une **Condition Si**, nom Any file loaded, reliée à la sortie verte de For each new file. **Expression** :

```
@not(empty(variables('last_file')))
```

POURQUOI TESTER LA VARIABLE ET NON LE COMPTEUR DU FILTER

Le compteur dit combien de fichiers étaient candidats ; la variable dit combien ont réellement été chargés. Si le dernier Copy avait échoué, l'exécution serait déjà en échec, mais la règle reste saine : on n'écrit la mémoire que sur une preuve.

9 Save watermark

SCRIPT · TRUE

- 1 Cliquez sur le + de la case **True**, choisissez **Script**, nom Save watermark.
- 2 **Paramètres** : connexion **Entrepôt**, WH_Control. Type de script **NonQuery**. Script, en contenu dynamique :

```
UPDATE dbo.watermark
SET last_file = '@{variables('last_file')}',
updated_at = GETDATE()
WHERE pipeline = 'PL_ColdChain_Express'
```

UNE LIGNE, ET LE PIPELINE SE SOUVIENT La prochaine exécution lira ce nom dans Get watermark et le Filter écartera tout ce qui est antérieur ou égal.

10 Send report

OUTLOOK · TRUE

- 1 Toujours dans la case **True**, ajoutez **E-mail Office 365**, nom Send report, **sans le relier** à Save watermark : les deux activités partent en parallèle, côte à côte. Connectez votre compte, renseignez **A**.

- 2 **Objet**, en contenu dynamique :

```
Chaîne du froid : @{activity('Keep new files').output.FilteredItemCount} fichier(s) chargé(s), dernier @{variables('last_file')}
```

- 3 **Corps** : tapez le texte au clavier, puis insérez la valeur par l'éclair, onglet **Fonctions**, sans le @ de tête :

```
Dernier fichier chargé : variables('last_file')
```

POURQUOI CÔTE À CÔTE Dans une branche, seule la première activité voit les activités du canevas principal. Un e-mail placé derrière le Script ne pourrait plus citer Keep new files. Deux premières activités, deux vues complètes.

11 Send nothing new

OUTLOOK · FALSE

- 1 Cliquez sur le + de la case **False**, **E-mail Office 365**, nom Send nothing new.
- 2 **Objet** : Chaîne du froid : aucun nouveau fichier. **Corps**, au clavier : « Le dossier landing ne contient rien de postérieur au dernier fichier chargé. Table et filigrane inchangés. »

DEUX FINS, TOUTES DEUX PROPRES Un pipeline qui n'a rien à faire doit le dire, pas se taire. **Valider**, puis **Enregistrer**.

Ce que contiendront le lakehouse et l'entrepôt

LH_ColdChain

Files / landing

readings_2026-10-04.csv · 05 · 06 · 07 · 08 (déposés par vous, jamais déplacés)

Tables / readings

5 748 lignes après la 3e exécution, colonne source_file, mode Append

WH_Control

dbo.watermark

PL_ColdChain_Express · readings_2026-10-08.csv · horodatage

Les fichiers restent dans **landing** : rien n'est déplacé, rien n'est supprimé. C'est le filigrane, et lui seul, qui sépare le passé du nouveau. La table **readings** grandit d'un fichier à la fois, et la ligne de **dbo.watermark** raconte à tout moment où en est le pipeline : quel fichier, à quelle heure.

Exécuter et vérifier

1 · LIRE ET LISTER

2 · FILTRER

3 · CHARGER

4 · MÉMORISER

5 · EXÉCUTER

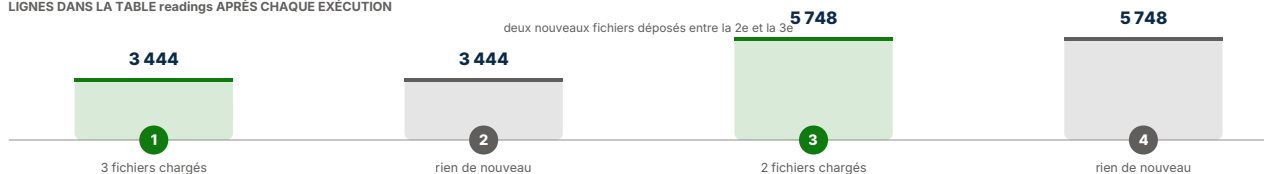
OBJECTIF

Quatre exécutions : le premier chargement, la relance à vide, l'arrivée de nouveaux fichiers, la relance à vide à nouveau.

RÉSULTAT

Les chiffres ci-dessous, à l'unité près.

LIGNES DANS LA TABLE readings APRÈS CHAQUE EXÉCUTION



Exécution 1 trois fichiers dans landing, filigrane vide

Exécuter. Onglet **Sortie** : Keep new files renvoie FilteredItemsCount = 3 ; les trois Load file donnent 1 152, 1 152 et 1 140. Branche True. E-mail : « Chaîne du froid : 3 fichier(s) chargé(s), dernier readings_2026-10-06.csv ». Dans l'entrepôt, SELECT * FROM dbo.watermark montre ce nom et l'heure.

Exécution 2 rien n'a changé

Keep new files renvoie 0, la boucle ne tourne pas, branche False, e-mail « aucun nouveau fichier ». La table est toujours à 3 444. Relancez autant de fois que vous voulez : rien ne bouge.

Exécution 3 après avoir déposé les fichiers du 7 et du 8 octobre dans landing

Le Get Metadata voit cinq fichiers, le Filter n'en garde que deux. Deux Load file à 1 152, table à 5 748, filigrane readings_2026-10-08.csv, e-mail « 2 fichier(s) chargé(s) ».

Exécution 4 rien n'a changé

Branche False à nouveau. Cinq fichiers dans le dossier, aucun n'est relu : toujours 5 748.

Vérifier dans le point de terminaison SQL

Ouvrez LH_ColdChain, basculez sur **Point de terminaison d'analyse SQL**, nouvelle requête :

```
SELECT site, COUNT(*) AS lignes,
SUM(CASE WHEN status = 'alert' THEN 1 ELSE 0 END) AS alertes,
SUM(CASE WHEN door_open = 'yes' THEN 1 ELSE 0 END) AS portes,
COUNT(DISTINCT source_file) AS fichiers
FROM readings GROUP BY site ORDER BY site;
```

SITE	LIGNES	ALERTES	PORTES	FICHIERS
AUGNY	1 920	12	63	5
ENNERY	1 908	7	58	5
WOIPPY	1 920	7	65	5

5 748

lignes dans la table

26

relevés hors plage

186

relevés porte ouverte

5 / 5

fichiers chargés, chacun une seule fois

CE QUE CES CHIFFRES PROUVENT Après l'exécution 1, la même requête donne 1 152, 1 140 et 1 152 lignes avec 3 fichiers par site. Après l'exécution 3, 5 fichiers par site et pas une ligne de plus que la somme des fichiers : aucun fichier n'a été relu, et le pipeline n'a jamais reçu de date.

Expressions et débannage

Les expressions du pipeline

OÙ	EXPRESSION
Get watermark, Requête	<code>SELECT last_file FROM dbo.watermark WHERE pipeline = 'PL_ColdChain_Express'</code>
Keep new files, Éléments	<code>@activity('List landing').output.childItems</code>
Keep new files, Condition	<code>@and(endswith(item().name, '.csv'), greater(item().name, activity('Get watermark').output.firstRow.last_file))</code>
For each new file, Éléments	<code>@activity('Keep new files').output.Value</code>
Load file, nom de fichier source	<code>@{item().name} ; colonne supplémentaire source_file = @{item().name}</code>
Remember file, Valeur	<code>@item().name</code>
Any file loaded, Expression	<code>@not(empty(variables('last_file')))</code>
Save watermark, Script	<code>UPDATE dbo.watermark SET last_file = '{variables('last_file')}', updated_at = GETDATE() WHERE pipeline = 'PL_ColdChain_Express'</code>
Send report, Objet	<code>Chaîne du froid : @{activity('Keep new files').output.FilteredItemsCount} fichier(s) chargé(s), dernier @{variables('last_file')}</code>

DEUX FAÇONS D'ÉCRIRE Dans un champ texte (nom de fichier, script, objet), l'expression s'insère entre `@{ }` au milieu du texte. Dans un champ expression (Éléments, Condition, Expression, Valeur d'une variable), elle commence par `@` et occupe tout le champ.

LA RÈGLE DU JEU DU FILIGRANE PAR NOM Elle tient tant que les noms sont croissants dans le temps, ce que garantit une date au format AAAA-MM-JJ. Un fichier corrigé doit porter un nom nouveau, postérieur au filigrane, sinon il est ignoré : c'est le prix d'une mémoire d'une seule ligne, et il est connu d'avance.

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Le <code>CREATE TABLE</code> refuse <code>DATETIME2</code>	Précision 7 non prise en charge par l'entrepôt. Écrivez <code>DATETIME2(6)</code> .
Get watermark ne renvoie rien	La ligne d'amorçage manque. Rejouez l' <code>INSERT</code> de l'étape 1 dans l'entrepôt.
Le Filter garde tout, même après un chargement	La casse de <code>last_file</code> ou de <code>firstRow</code> diffère, ou le Script n'a pas tourné. Vérifiez <code>SELECT * FROM dbo.watermark</code> .
Le Filter ne garde rien à la première exécution	Le filigrane n'est pas vide (espace, ancienne valeur). Remettez-le à <code>' '</code> par un <code>UPDATE</code> .
« property 'Value' doesn't exist »	Le <code>V</code> est une majuscule : <code>output.Value</code> , et <code>output.FilteredItemsCount</code> .
Load file : « PathNotFound »	Le dossier est <code>landing</code> sans barre oblique, et le nom de fichier est <code>@{item().name}</code> avec les accolades.
La variable finit avec le mauvais fichier	Le <code>ForEach</code> n'est pas Séquentiel . Cochez la case.
« Impossible de référencer la sortie de l'activité... pas un ancêtre »	L'activité est placée derrière une autre dans une branche. Mettez-la en première position, côte à côte.
Le Script échoue sur l'apostrophe	Le script est en contenu dynamique et la valeur est entre apostrophes simples : <code>'{variables('last_file')}'</code> . Vérifiez les deux apostrophes.
Le texte ne se colle pas dans le corps de l'e-mail	Tapez-le au clavier ; les valeurs dynamiques passent par l'éclair, onglet Fonctions.
Valider et Exécuter restent grisés, un rond tourne	Enregistrement en cours. Patientez ; ne rechargez la page qu'après avoir vu le point disparaître.